TALLER DE METODOLOGÍAS ÁGILES · 2026-2 · TERCER CORTE

Punto de control del proyecto final

Programar en pareja con una persona o con un agente: qué se gana, qué se pierde y qué aprendizaje desaparece

TEMA DE EXPOSICIÓN

3 · Pair programming

PRÁCTICA QUE ASISTE EL AGENTE

Desarrollo de una tarea con el agente como copiloto

PAREJA

Ana Sofía Arango Yanza · Juan José Arias Chacua

PROFESOR

César Jesús Pardo Calvache, Ph.D.

FECHA Y VERSIÓN

Viernes 30 de octubre de 2026 · v1 (borrador del 26/10)

REPOSITORIO

github.com/Sofii141/CAMEIA-lite · ramas `main` (código) y `documentacion`

0 Estado frente a lo que pide el punto de control

El punto de control revisa tres cosas: el caso base, la especificación del agente (v1) y la línea base ya medida. Además, ese día se intercambian tres casos con otra pareja.

CASO BASE

24 casos

Listo 10 difíciles · congelar el 26/10

ESPECIFICACIÓN V1

1 agente

Borrador falta fijar la herramienta y el modelo

LÍNEA BASE

≥ 10 tareas

Se mide el 29/10 pareja humana, sin IA

Requisito del enunciado	Dónde está	Estado
Caso base con ≥ 10 casos e incluye material difícil	§2 · repo `main`	Listo
Caso congelado antes de tocar el agente	commit `congelado v1`	26/10
Línea base medida con GQM, ≥ 10 casos y condiciones registradas	§3 y §5	29/10
Especificación de agentes v1 con sus 8 campos	§6	Borrador
Tres casos para entregar a otra pareja y tres que recibimos	`02 - Caso base/Casos …`	30/10

ANTES DE ENVIAR ESTE DOCUMENTO

Todo lo marcado Pendiente se completa con datos reales. Las tablas de la línea base (§5) están **vacías a propósito**: no se llenan hasta medir. Inventar datos de línea base es lo único que invalida el trabajo completo.

1 Tema, práctica e hipótesis

El enunciado asigna al tema 3 la práctica **"desarrollo de una tarea con el agente como copiloto"**, y como métricas sugiere tiempo, retrabajo y errores detectados a tiempo. Para que el proyecto cuente una sola historia con la exposición, agregamos la pregunta central del tema: **qué aprendizaje se pierde** cuando el compañero es un agente que no necesita entender.

Qué dice la literatura (y qué esperamos ver)

Hallazgo publicado	Hipótesis para nuestro caso
Hannay et al. 2009 (metaanálisis, *IST*): el pairing humano tiene un efecto pequeño en calidad, medio en duración y negativo en esfuerzo, y depende de la complejidad.	H1: con el agente, la duración baja en las tareas S y M, pero no en las difíciles.
METR 2025 (RCT, 16 devs): con IA fueron 19 % más lentos, aunque creían ir ~20 % más rápido.	H2: la velocidad *percibida* con el agente supera a la medida.
Welter et al. 2025 (ASE): las sugerencias de la IA se aceptan con menos escrutinio que las de un compañero humano.	H3: con el agente se escapan más errores a las pruebas ocultas.
Anthropic 2026 (RCT, 52 ingenieros): con IA, ~17 puntos menos de comprensión, sobre todo en depuración.	H4: el codewalk baja en la condición con agente, sobre todo para el compañero que no participó.

> **CÓMO LEEREMOS LOS RESULTADOS**
>
> Con n ≈ 10 por condición no podemos afirmar causalidad: reportaremos diferencias y tendencias, junto con lo que **no** demuestran. Si el agente no mejora la práctica, lo diremos con los datos.

2 El caso base: CAMEIA-lite

24 tareas pequeñas de programación. Cada una aísla una regla **real** del backlog de CAMEIA, el proyecto en el que trabajamos los dos durante el semestre.

De dónde sale

De los 324 PR fusionados en los 9 repositorios de CAMEIA (extraídos con `gh` el 25/10). Cada tarea viene de un ítem de Jira (17 en total, de 5 repos) y de su PR. La trazabilidad está en `26102026_01_Backlog_Origen_CAMEIA-lite_v1.xlsx`.

Por qué así y no directamente sobre CAMEIA

Las tareas reales exigen montar Docker, Firebase y PostgreSQL, y sus PR son muy desiguales en tamaño. Así, el cronómetro mediría la configuración y no la práctica. CAMEIA-lite conserva los requisitos reales, se corre con un solo comando y permite tareas comparables entre condiciones.

Qué lo hace representativo y difícil

- **11 S · 11 M · 2 L**, asignadas por sorteo estratificado.
- **10 difíciles**: Unicode/NFC, 29 de febrero, concurrencia por doble clic, orden de reglas 404/409, inyección de cabeceras, el subdominio trampa en CORS, la variable vacía que sí existe y el falso obvio `production` / `prod`.
- Cada tarea trae **pruebas visibles** (las que guían el desarrollo) y **pruebas ocultas** (las que miden los errores escapados).
- Validado: las 110 pruebas pasan con soluciones de referencia y fallan todas con el esqueleto.

Las 24 tareas

ID	Tarea	Tam.	Difícil	Origen (Jira)	Ya la hizo
`t01`	Normalizar texto	S		CM-36	—
`t02`	Contar caracteres	S		CM-36	—
`t03`	Validar nombre	M	Difícil	CM-36	—
`t04`	Validar correo	S		CM-36	—
`t05`	Validar celular	S		CM-36	—
`t06`	Política de contraseñas	M		CM-36	—
`t07`	Fecha de nacimiento	M	Difícil	CM-279	—
`t08`	Cupo de perfiles por plan	M	Difícil	CM-271	—
`t09`	Nivel de log por estado HTTP	S		CM-184	—
`t10`	Mensajes de error por campo	M		CM-336 / CM-298	Juan José

ID	Tarea	Tam.	Difícil	Origen (Jira)	Ya la hizo
t11	Contador de reintentos	M		CM-217	Juan José
t12	Crear una sola vez (doble clic)	L	Difícil	CM-270	Juan José
t13	Eliminar rol objetivo	M	Difícil	CM-71 / CM-271	—
t14	Sustituir rol conservando su lugar	S		CM-72	—
t15	Catálogo de roles por idioma	S		CM-82 / CM-72	Ana Sofía
t16	Fechas de formación	M		CM-271	—
t17	Expectativa salarial	S		CM-271	—
t18	Propagar X-Request-Id	M	Difícil	CM-279 / CM-104	—
t19	Limpiar cabeceras de identidad	M	Difícil	CM-104	—
t20	Origen permitido (CORS)	S	Difícil	CM-162	—
t21	Perfil de arranque y emulador	M	Difícil	CM-190 / CM-191 / CM-175	—
t22	Registro idempotente (Idempotency-Key)	L	Difícil	CM-340	—
t23	Progreso de la entrevista	S		CM-217	Juan José
t24	Enmascarar correo en logs	S		CM-271 (registro sin datos personales)	—

LIMITACIONES DECLARADAS

- Las reglas se sacan de su entorno, así que no medimos el costo de integrar con el sistema real.
- Los PR originales se escribieron con Claude Code, de modo que la solución de referencia puede tener "estilo de agente".
- Cinco tareas ya las implementó uno de nosotros: esas se asignan a la otra persona y solo en la condición con agente.
- Cinco reglas se ajustaron para hacer el caso más difícil y van marcadas `[P]` en el backlog de origen.

3 GQM

Objetivo: analizar el desarrollo de una tarea con un agente copiloto, frente al pair programming humano, para evaluar su efecto en tiempo, esfuerzo, errores escapados, retrabajo y comprensión del código, en las 24 tareas de CAMEIA-lite.

Pregunta	Métrica	Cómo se toma	Qué NO representa	Respaldo
Q1 ¿Se termina más rápido?	M1 Duración (min)	Cronómetro hasta pasar las pruebas visibles	Productividad general	Hannay 2009; Shihab 2025
Q2 ¿Cuesta más persona-tiempo?	M2 Esfuerzo = M1 × personas	Calculado (pareja = 2)	Tiempo de aprender la herramienta	Hannay 2009
Q3 ¿Se escapan más errores?	M3 Pruebas ocultas que fallan	`pytest tests/ocultas -k tNN`	Defectos de diseño o legibilidad	Arisholm 2007
Q4 ¿Hay más retrabajo?	M4 Minutos y commits después de "terminado"	git log + cronómetro	El tamaño de cada corrección	Enunciado del curso
Q5 ¿Qué aprendizaje se pierde?	M5 Codewalk 0–5 (autor y compañero)	5 preguntas fijas, sin mirar el código	La habilidad general	Welter 2025; Anthropic 2026
Q6 ¿Nos engaña la sensación de velocidad?	M6 Pronóstico antes y ayuda percibida después	Hoja de medición	El resultado real	METR 2025

Las 5 preguntas del codewalk

1. ¿Qué hace la función, en una frase y sin leerla?
2. ¿Qué caso borde maneja y dónde?
3. ¿Por qué se eligió esta forma y no otra?
4. Si mañana falla con la entrada X, ¿dónde mirarías primero? *(depuración)*
5. ¿Qué cambiarías si el requisito cambia en Y?

4 Diseño de la medición

	Línea base (antes del 30/10)	Ciclo 1 (2–6 nov)	Ciclo 2 (9–13 nov)
Condición	Pareja humana: conductor y navegante, cambian cada 15 min, sin IA	Una persona + el agente copiloto; el compañero no participa	
Tareas	10 (por sorteo)	6	6 + los 3 casos recibidos
Evidencia	Grabación de pantalla, commits por tarea, hoja de medición y bitácora (en los ciclos, una fila por cada uso del agente)		

Protocolo por tarea

1. Anotar el pronóstico de minutos.
2. Iniciar el cronómetro al abrir el enunciado.
3. Trabajar solo con las pruebas visibles.
4. Declarar "terminado" cuando pasan, parar el cronómetro y hacer commit `tNN: terminado`.
5. Correr las pruebas ocultas: cada fallo es un error escapado. Si se corrige, cuenta como retrabajo.
6. Codewalk de 5 minutos al autor y al compañero.
7. Llenar la hoja de medición al momento, no al final del día.

CONTROLES PARA QUE LA COMPARACIÓN VALGA

- **Sorteo reproducible** (`python sorteo.py <semilla>`): la semilla queda en un acta.
- **Estratificación por tamaño**: cada condición recibe una mezcla parecida de S, M y L.
- Si alguien abre las **pruebas ocultas antes de tiempo**, la tarea se descarta con la marca `[C]`.
- En los ciclos, la carpeta `tests/ocultas` se saca del espacio de trabajo del agente.

5 Línea base

Se mide el jueves 29/10

Condiciones de la medición

Campo	Valor
Participantes	Ana Sofía Arango Yanza y Juan José Arias Chacua
Fecha, hora y duración total	
Lugar o canal	
Herramientas	Python 3.13, pytest, editor _____ · **sin IA**
Semilla del sorteo	
Interrupciones o incidentes	

Resultados por tarea

Tarea	Tam.	Pronóstico (min)	Duración (min)	Esfuerzo (min·pers.)	Ocultas que fallan	Retrabajo (min)	Codewalk A / J

Resumen (lo recalcula de forma independiente la otra persona)

Métrica	S	M	L	Total	Recalculó
Duración media					
Errores escapados (total)					
Codewalk medio					

6 Especificación del agente · v1

Campo	Contenido
Rol y alcance	Navegante en el desarrollo de *una* tarea de CAMEIA-lite: propone enfoque, código y pruebas. La persona conduce y decide.
Entradas	El enunciado y los criterios de aceptación de la tarea (`TAREAS.md`), su archivo `src/cameia_lite/tNN.py` y las pruebas visibles. Nada más.
Salidas	Un plan corto en pasos, el diff propuesto y la lista de supuestos que hizo.
Herramientas y permisos	Herramienta y modelo: **por definir**. Puede leer y escribir solo en el archivo de la tarea, en una rama local. No tiene acceso a `tests/ocultas` , al remoto ni a credenciales.
Prohibiciones	No hacer push ni merge. No modificar pruebas ni otros archivos. No inventar requisitos que no estén en el enunciado. No desactivar ni saltarse pruebas. No leer `tests/ocultas`.
Criterio de validación	Se acepta solo si pasa las pruebas visibles, cumple los criterios de aceptación y quien conduce puede explicar cada línea sin mirar al agente.
Punto de control humano	Antes de cada commit, quien conduce decide aceptar, corregir o rechazar y lo registra en la bitácora con su razón. Después, el compañero hace el codewalk.
Versión	v1 · 25/10/2026. Cada cambio da una versión nueva, con el dato de la bitácora que lo motivó.

Empezamos con un solo agente. Solo agregaremos un segundo ("revisor") en el ciclo 2 si la bitácora del ciclo 1 muestra un problema que el primero no resuelve.

7 Calendario

29 oct
Línea base: 10 tareas en pareja humana

30 oct
Punto de control e intercambio de 3 casos

2–6 nov
Ciclo 1 con el agente, con bitácora y spec v2

9–13 nov
Ciclo 2, 3 fallos documentados y video

16–18 nov
Reporte, diapositivas y exposición

Preguntas para el profesor en el punto de control

☐ ¿Acepta CAMEIA-lite (requisitos reales de CAMEIA, aislados como funciones) como caso base, en lugar de trabajar directamente sobre los repositorios?

☐ ¿Es válido agregar la comprensión del código (codewalk) a las métricas sugeridas para el tema 3?

☐ ¿Con qué pareja intercambiamos los tres casos?

8 Fuentes

- Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology*, 51(7), 1110–1122. doi:10.1016/j.infsof.2009.02.001
- Arisholm, E., Gallis, H., Dybå, T., & Sjøberg, D. I. K. (2007). Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE TSE*, 33(2), 65–86. doi:10.1109/TSE.2007.17
- Welter, A. et al. (2025). From developer pairs to AI copilots: A comparative study on knowledge transfer. ASE 2025. arXiv:2506.04785
- Shihab, M. I. H., Hundhausen, C. et al. (2025). The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield programming tasks. arXiv:2506.10051
- Becker, J., Rush, N., Barnes, B., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. METR. arXiv:2507.09089
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590
- Ma, Q., Wu, T., & Koedinger, K. (2023). Is AI the better programming partner? Human-human pair programming vs. human-AI pAIr programming. LLM@AIED. arXiv:2306.05153
- Anthropic (2026). How AI assistance impacts the formation of coding skills. anthropic.com/research
- Basili, V., Caldiera, G., & Rombach, H. D. (1994). The Goal Question Metric approach.

9 Declaración de uso de IA

Herramienta	Para qué	Qué verificamos nosotros
Claude Code	Leer los enunciados y la retroalimentación de la E1, proponer la estructura, generar las 24 tareas con sus pruebas, armar el backlog de origen a partir de los PR, buscar fuentes y maquetar este documento.	**Completar** que revisamos las 24 tareas, que cada fila del backlog coincide con su PR y que leímos las fuentes citadas.

Ningún dato de línea base ni de bitácora fue generado por IA: esos datos salen solo de nuestras sesiones cronometradas.